

Code-First Approach in ASP.NET Core with Entity Framework Core

Step 1: Install Entity Framework Core

To use EF Core, install the necessary packages via NuGet Package Manager Console:

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer
```

```
Install-Package Microsoft.EntityFrameworkCore.Tools
```

```
Install-Package Microsoft.EntityFrameworkCore.Design
```

```
Install-Package Microsoft.EntityFrameworkCore
```

```
Install-Package Microsoft.Data.SqlClient
```

Step 2: Create the Model Classes

Create a Models folder and define your entity class. Example:

using System.ComponentModel.DataAnnotations;

```
public class Product
{
    [Key] // Primary Key
    public int ProductId { get; set; }

    [Required]
    public string Name { get; set; }

    public decimal Price { get; set; }
}
```

Step 3: Create the DbContext

Create a new ApplicationDbContext class inside a Data folder:

using Microsoft.EntityFrameworkCore;

```
public class ApplicationDbContext : DbContext
```

```

{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
    base(options) { }

    public DbSet<Product> Products { get; set; } // Table in DB
}

```

Step 4: Configure Database Connection

Modify appsettings.json and add the connection string:

```

"ConnectionStrings": {
    "DefaultConnection":
    "Server=YOUR_SERVER_NAME;Database=YourDatabase;Trusted_Connection=True;MultipleActiveResultSets=true"
}

```

Step 5: Register DbContext in Program.cs

Modify Program.cs:

```

using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

// Configure Database Context
builder.Services.AddDbContext<ApplicationDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

```

```
app.UseHttpsRedirection();  
app.UseStaticFiles();  
app.UseRouting();  
app.UseAuthorization();  
  
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");  
  
app.Run();
```

Step 6: Create Migrations

Run the following command in **Package Manager Console (PMC)**:

```
Add-Migration InitialCreate
```

This will generate a **Migrations** folder with migration scripts.

Step 7: Apply Migrations to Create Database

Execute:

```
Update-Database
```

This applies the migration and creates the database.

Step 8: Create Controller with CRUD Operations

Run the following command to generate a controller:

```
Scaffold-Controller -Name ProductController -Model Product -DataContext  
ApplicationDbContext -ScaffoldDbContext
```

Alternatively, manually create ProductController:

```
using Microsoft.AspNetCore.Mvc;  
using Microsoft.EntityFrameworkCore;
```

```
public class ProductController : Controller
{
    private readonly ApplicationDbContext _context;

    public ProductController(ApplicationDbContext context)
    {
        _context = context;
    }

    public async Task<ActionResult> Index()
    {
        return View(await _context.Products.ToListAsync());
    }

    public IActionResult Create()
    {
        return View();
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<ActionResult> Create([Bind("ProductId,Name,Price")] Product
product)
    {
        if (ModelState.IsValid)
        {
            _context.Add(product);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(Index));
        }
    }
}
```

```
    }  
    return View(product);  
}  
}
```

Step 9: Run the Application

Run the project (Ctrl + F5), navigate to:

<https://localhost:xxxx/Product>

You should see the product list.

Additional Commands

- **View Applied Migrations:**

Get-Migrations

- **Rollback Last Migration:**

Remove-Migration

- **Apply Pending Migrations:**

Update-Database

This completes the **Code-First** approach in ASP.NET Core MVC with EF Core.